CLOUD COMPUTING

Web Services with SOAP and Service Registry Integration

CLIMATE-AWARE RESTAURANT FINDER

A Distributed, Live-Data SOAP-Based Restaurant Recommendation Service

Name	[Your Name]
Register Number	[Your Register Number]
Programme	[Your Programme]
Subject	Cloud Computing

Technologies Used

Java 17 · Spring Boot · SOAP · Eureka Service Registry · Thymeleaf · Open-Meteo Live Weather API · Browser Geolocation

1 Abstract

The Climate-Aware Restaurant Finder is a distributed web application developed using Java, Spring Boot, SOAP and Netflix Eureka. The system helps a user find a nearby restaurant that is best suited to the current climate and temperature of a given location, using **live weather data** and the user's **real-time browser location** rather than static or simulated inputs.

The application consists of four independent Spring Boot services: a Gateway Web Application, a Weather Service, a Restaurant Service and a Eureka Service Registry. The Gateway obtains the user's latitude/longitude automatically from the browser's geolocation API (with manual entry as a fallback), discovers the Weather Service and Restaurant Service through the registry, and communicates with both using SOAP. The Weather Service calls a live, key-less weather provider (Open-Meteo) to fetch the current temperature, humidity and precipitation for those exact coordinates and classifies the climate (HOT, WARM, MILD, COLD, HUMID or RAINY) in real time. The Restaurant Service returns restaurants ranked by distance and by how well each restaurant suits that climate.

The project demonstrates contract-first SOAP web services, service discovery using Eureka, XML Schema (XSD) driven code generation, live third-party API integration, and orchestration of multiple SOAP services from a single gateway application.

2 Introduction

Choosing a restaurant that is actually comfortable to visit often depends on the weather at that moment – an air-conditioned indoor restaurant is preferable on a hot, humid afternoon, while a cozy indoor restaurant with warm food is preferable on a cold or rainy day. This project implements a distributed restaurant recommendation system that takes a location – detected automatically from the user's device wherever possible – determines the *actual current* climate at that location using a live weather feed, and recommends the nearest restaurant that is well suited to that climate.

The application does not maintain a single monolithic list of restaurants tied to weather conditions, nor does it rely on a fixed or simulated weather value. Instead, two independent SOAP services are combined by a gateway: one service is solely responsible for fetching and classifying live climate data, and the other is solely responsible for ranking restaurants by distance and climate suitability. This separation of concerns mirrors how real distributed cloud services are composed.

2.1 Objectives

- Develop a distributed application using Java and Spring Boot.
- Implement multiple independent, contract-first SOAP Web Services.
- Demonstrate service discovery using a Eureka service registry.
- Generate SOAP request/response classes from XML Schema (XSD) contracts.
- Dynamically resolve service endpoints at runtime instead of hardcoding host/port values.
- Fetch **live, real-world weather data** for the exact coordinates supplied, instead of a simulated value.
- Automatically detect the user's **current location** via the browser, instead of relying on a fixed default.
- Recommend a nearby restaurant that is well suited to the current, real climate.
- Present the recommendation through a simple web interface and a JSON API.

3 System Architecture

The application follows a distributed, service-oriented architecture. Three Spring Boot services have separate responsibilities and register themselves with a central Eureka service registry, while the gateway web application discovers and orchestrates them. The Weather Service, in turn, depends on an external live data source (Open-Meteo) over plain HTTPS.

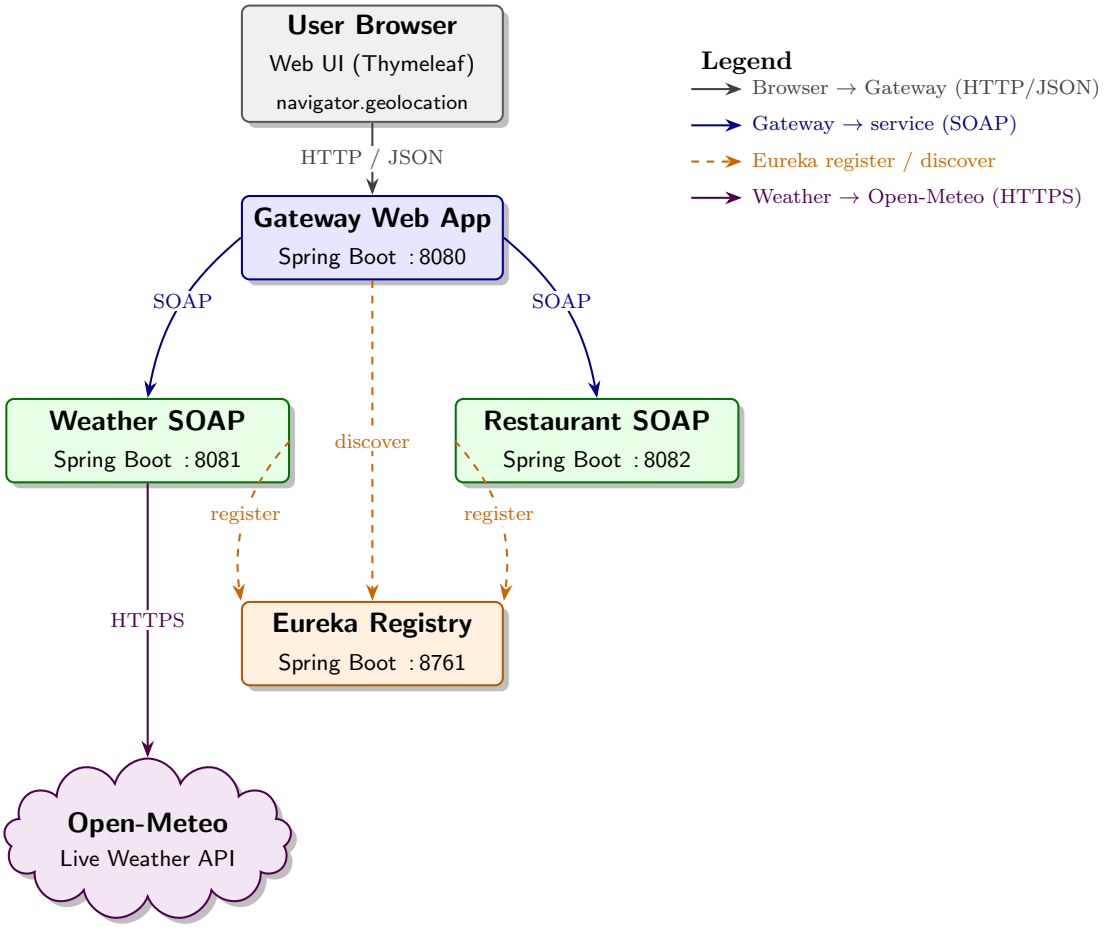


Figure 1: System Architecture – client, gateway, registry, SOAP services and the external live-weather provider

The colours group the components by role: grey for the client-facing web UI, blue for the Gateway that orchestrates every request, green for the two independent SOAP microservices, orange for the Eureka registry both of them register with, and violet for the external Open-Meteo API that the Weather Service depends on. Dashed orange arrows are Eureka registration/discovery traffic; solid blue arrows are SOAP calls; the violet arrow is the one plain HTTPS call this system makes to a third party.

The browser communicates with the Gateway Web Application, which coordinates the complete request. On first load, the page asks the browser for the user's current GPS coordinates (`navigator.geolocation`); if the user allows it, the page reloads with those exact coordinates. If geolocation is denied or unsupported, the user can still enter coordinates manually. The Gateway then discovers the Weather Service and the Restaurant Service through the Eureka Registry and invokes both using SOAP.

The Weather Service calls the Open-Meteo API for the given coordinates and classifies the climate from the live response. The Restaurant Service retrieves candidate restaurants from its catalog, computes the distance to each using the Haversine formula, scores each restaurant for suitability against the given climate type, and returns a ranked list. The Eureka Registry

provides the Weather Service and Restaurant Service endpoints to the Gateway, so no internal service address is hardcoded anywhere in the Gateway's source code.

All four Spring Boot services can run on a single machine for development, or be distributed across multiple hosts/VMs (e.g. a cloud virtual machine) without any code change, since every inter-service address is resolved through the registry at runtime.

4 Data Flow

The complete request processing flow is:

1. The browser requests the user's current location via the Geolocation API (or the user enters latitude/longitude manually).
2. The browser sends the request, with those coordinates, to the Gateway Web Application.
3. The Gateway queries the Eureka Registry for `weather-service`.
4. The Registry returns the current Weather Service endpoint.
5. The Gateway sends a SOAP request (`getClimateRequest`) containing the coordinates.
6. The Weather Service calls the live Open-Meteo API for those coordinates, classifies the response, and returns the current temperature and climate classification over SOAP.
7. The Gateway queries the Eureka Registry for `restaurant-service`.
8. The Registry returns the current Restaurant Service endpoint.
9. The Gateway sends a SOAP request (`getRestaurantsRequest`) containing the coordinates and the climate type obtained in step 6.
10. The Restaurant Service filters its catalog, computes distance and climate-match score for each restaurant, and returns a ranked list.
11. The nearest, best-climate-fit restaurant is highlighted and the full ranked list is returned to the browser.

The overall flow can be summarized as:

Browser (geolocation) → Gateway → Eureka Registry → Weather Service → Open-Meteo API → Weather Service → Gateway → Eureka Registry → Restaurant Service → Gateway → Browser

The system does not use a fixed, weather-blind restaurant list, nor a fixed weather value – the climate result is fetched live for the exact coordinates on every request, and the restaurant ranking depends on that live result within the same request.

5 SOAP Web Services

Both backend services expose contract-first SOAP endpoints described by an XML Schema (XSD); Java request/response classes are generated from these schemas at build time.

5.1 Weather Service

`http://<host>:8081/ws` (WSDL at `/ws/weather.wsdl`)

The service accepts:

- Latitude
- Longitude

Internally, it calls the Open-Meteo `/v1/forecast` endpoint for `temperature_2m`, `relative_humidity_2m`, `precipitation` and `weather_code` at those coordinates, and classifies the result. If the live call fails (e.g. no network access), it falls back to a latitude-based estimate and clearly marks the result as estimated rather than live, so the two are never confused.

It returns:

- Temperature in degrees Celsius
- Climate type (HOT, WARM, MILD, COLD, HUMID, RAINY)
- A rain flag and a short human-readable summary, labelled as either live or estimated

5.2 Restaurant Service

`http://<host>:8082/ws` (WSDL at `/ws/restaurant.wsdl`)

The service accepts:

- Latitude and longitude
- Climate type
- Search radius (kilometres) and maximum number of results

and returns a list of restaurants, each with name, cuisine type, indoor-seating/air-conditioning flags, distance from the caller, a climate-match score, and a short description.

SOAP acts as the abstraction layer between the Gateway and each backend service: the Gateway receives a structured, strongly-typed SOAP response rather than working with an ad-hoc format. This demonstrates the use of SOAP for communication between independent application services, even where one of those services (Weather) is itself backed by an external live REST/JSON API.

6 Service Registry

A Eureka Service Registry is used to demonstrate service discovery. It runs on port 8761 and stores the registered instance information (host, port, status) for every service.

The Weather Service and Restaurant Service register themselves under the application names `weather-service` and `restaurant-service`.

Mechanism	Purpose
Eureka Client (auto-registration)	Each SOAP service registers its host/port on startup
`DiscoveryClient.getInstances(id)`	Gateway looks up a live instance by application name
Eureka Dashboard (:8761)	Visual confirmation that both services are UP

The Gateway queries the registry before every SOAP call rather than depending on a fixed configuration, so it continues to work if a service restarts on a different host or port. This demonstrates service discovery without hardcoding downstream endpoints in the Gateway.

7 Live Weather Integration

Unlike a simulated or deterministic weather value, the Weather Service now sources its data from Open-Meteo (https://open-meteo.com), a free, key-less weather API, at request time.

- **Request**: for the given latitude/longitude, the service calls `GET /v1/forecast?latitude=..&longitude`
- **Classification**: the returned temperature, humidity and WMO weather code are mapped onto the existing `ClimateType` contract (HOT, WARM, MILD, COLD, HUMID, RAINY), so no downstream contract or client code needed to change.
- **Resilience**: if the live call fails for any reason, the service falls back to a latitude-based estimate rather than failing the whole request, and the summary text explicitly states whether the reading is live or estimated.

This keeps the recommendation genuinely responsive to real-world conditions: the same coordinates queried on a hot afternoon and a rainy evening will correctly produce different climate classifications, and therefore potentially different restaurant recommendations.

8 Live Location Detection

The Gateway's web interface no longer depends on a fixed default location. On first visit, it uses the browser's `navigator.geolocation.getCurrentPosition()` API to request the user's

current coordinates and automatically reloads the page with them. A manual latitude/longitude form and a "Use my current location" button remain available so the feature degrades gracefully if the user declines the permission prompt or their browser does not support geolocation.

9 Recommendation Logic

After receiving the live climate classification and the candidate restaurant list through SOAP, the Restaurant Service (and, for the final ordering shown to the user, the Gateway) processes the available restaurants.

The recommendation process is:

1. Compute the distance from the user's coordinates to every restaurant in the catalog.
2. Keep restaurants within the search radius (falling back to the full catalog if none are within range).
3. Score each restaurant for how well it fits the given climate type – restaurants explicitly tagged for that climate score highest, with partial credit given for related attributes such as air-conditioning on hot/humid days or indoor seating on cold/rainy days.
4. Sort primarily by climate-match score, then by distance.
5. Present the top result as the recommended restaurant, with the remaining ranked results shown alongside it.

Distance is calculated using the Haversine formula:

$$d = 2R \arcsin \sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)} \tag{1}$$

where R represents the approximate radius of the Earth, φ_1, φ_2 are the latitudes of the two points, and $\Delta\varphi, \Delta\lambda$ are the differences in latitude and longitude respectively.

The final response contains the restaurant name, cuisine type, distance, climate-match score, seating/AC attributes and a short description for each ranked restaurant.

10 Tools and Technologies

The following tools and technologies were used during development and testing.

Tool / Technology	**Purpose**
Java 17	Application development
Spring Boot	Development of the distributed services
Spring Web Services (Spring-WS)	SOAP Web Service implementation
Spring Cloud Netflix Eureka	Service registry and service discovery
JAXB / jaxb2-maven-plugin	Generating Java classes from XSD contracts
Open-Meteo API	Live, key-less weather and climate data
Java 17 `HttpClient` + Jackson	Calling and parsing the live weather API
Browser Geolocation API	Automatic detection of the user's current location
Thymeleaf	Server-rendered web interface
Maven	Build and dependency management
Web Browser	User interface

The four services were developed as separate Maven modules under a single multi-module parent project, built together with a single `mvn install` at the root.

11 Deployment

The four services are independent Spring Boot applications and can be run on a single development machine, or distributed across multiple hosts or virtual machines (for example, on a cloud provider such as Microsoft Azure or AWS) without any source-code changes, since every downstream address is resolved at runtime through the Eureka Registry rather than being hard-coded. The only external dependency introduced by the live-data changes is outbound HTTPS access from the Weather Service to Open-Meteo, and the browser's own network access for the Geolocation permission prompt (which requires the page be served over `localhost` or HTTPS).

Service	Port
Gateway Web App	8080
Restaurant SOAP Service	8082
Weather SOAP Service	8081
Eureka Registry	8761

Port 8080 provides the public entry point to the web application. The Weather Service, Restaurant Service and Eureka Registry are intended for internal communication only and would typically not be exposed publicly in a production deployment.

12 Implementation and Result

The four Spring Boot services were successfully implemented as a multi-module Maven project. Both the Weather Service and the Restaurant Service expose a SOAP WSDL generated from their respective XSD contracts. The Gateway resolves both services through the Eureka Registry, invokes them over SOAP in sequence, and combines their results.

The complete workflow was demonstrated through the web interface: the browser automatically supplies the user's current coordinates, the system fetches the live climate classification for that exact location, and returns a ranked list of nearby restaurants, with the restaurant that best matches both proximity and the real, current climate highlighted as the recommended choice. A JSON API endpoint (`/api/recommend`) is also available for programmatic access to the same result.

13 Limitations

The restaurant catalog used by the Restaurant Service is an in-memory, hardcoded dataset rather than a live external data source, so results are limited to the sample locations included in the catalog.

The climate classification produced by the Weather Service now uses live meteorological data from Open-Meteo rather than a simulated value; it only falls back to a deterministic latitude-based estimate if the live call fails (for example, no internet access), and that fallback is clearly labelled in the response so it is never mistaken for a live reading.

Automatic location detection depends on the user's browser and device supporting and granting permission for the Geolocation API, and requires the page to be served over `localhost` or HTTPS; manual latitude/longitude entry remains available as a fallback.

The Eureka Registry uses default single-instance settings, and the Gateway currently selects the first available instance of a service rather than load-balancing across multiple instances.

These limitations are acceptable for the scope of this mini project.

14 Conclusion

The Climate-Aware Restaurant Finder demonstrates a distributed Web Services application using Java, Spring Boot, SOAP and a Eureka-based service registry, extended with live, real-world data sources rather than static or simulated inputs.

The Gateway Web Application coordinates the complete request – starting from the user's own detected location – the Weather Service and Restaurant Service each expose an independent contract-first SOAP endpoint, and the Eureka Registry demonstrates service discovery without hardcoded endpoints. The Weather Service's integration with a live third-party API shows how a contract-first SOAP service can wrap an external REST/JSON data source without changing its own contract. The Restaurant Service combines distance-based ranking (Haversine formula) with a climate-suitability score to produce a recommendation that is both nearby and appropriate for the actual current weather.

Overall, the project provides a practical implementation of SOAP communication, XSD-driven contract-first development, service discovery, live external API integration, browser-based geolocation, and orchestration of multiple independent services from a single gateway application.

References

1. Spring Boot. https://spring.io/projects/spring-boot
2. Spring Web Services. https://spring.io/projects/spring-ws
3. Spring Cloud Netflix. https://spring.io/projects/spring-cloud-netflix
4. Netflix Eureka. https://github.com/Netflix/eureka
5. Jakarta XML Binding (JAXB). https://eclipse-ee4j.github.io/jaxb-ri/
6. Open-Meteo Weather API. https://open-meteo.com
7. MDN Web Docs – Geolocation API. https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API